C++ LEVEL 1

Revision Question Bank

40 practice problems • Easy to challenging • C++17

Topics covered
Input/output, conditions, loops, digit manipulation, number theory, and nested-loop patterns.
How to use this workbook
Attempt each question before looking up a solution. Compile with warnings enabled, test normal and boundary cases, and write down any mistakes you discover.
Compiler suggestion
`g++ -std=c++17 -Wall -Wextra -Wpedantic file.cpp -o program`

Name: _____ Date started: _____

Part A — Basic Input, Output and Conditions

1. Print Hello World

Print `Hello, World!` to the console.

2. Sum of Two Integers

Read two integers and print their sum.

3. Even or Odd

Determine whether an integer is even or odd.

4. Largest of Two Numbers

Read two numbers and print the larger number. Handle equality.

5. Largest of Three Numbers

Find the maximum of three numbers, including cases where two or more are equal.

6. Simple Calculator

Read two numbers and an operator (+, -, *, /). Perform the operation and handle division by zero.

7. Leap Year

Determine whether a year is a leap year using the Gregorian calendar rules.

8. Multiplication Table

Print the multiplication table of a given integer from 1 to 10.

Part B — Loops and Digit Manipulation

9. Sum of First N Natural Numbers

Read n and calculate 1 + 2 + ... + n using a loop.

10. Reverse an Integer

Reverse the digits of a non-negative integer without converting it to a string. Detect overflow.

11. Count the Digits

Count the digits of an integer using arithmetic and a loop. Zero has one digit. Bonus: handle negative integers without using std::abs().

12. Check for a Prime Number

Determine whether an integer is prime. Handle numbers less than 2 and avoid checking every divisor up to n - 1.

13. Factorial of a Number

Calculate n! using a loop. Handle 0! = 1 and consider integer overflow.

14. Fibonacci Sequence

Print the first n Fibonacci numbers using iteration.

15. Sum of Digits

Calculate the sum of all digits in an integer. Example: 583 → 16.

16. Armstrong Number

Check whether a non-negative integer equals the sum of its digits, each raised to the power of the number of digits. Example: 153 is an Armstrong number.

Part C — Number Properties

17. Palindrome Number

Check whether a non-negative integer reads the same forward and backward. Example: 1221.

18. Greatest Common Divisor (GCD)

Find the GCD of two non-negative integers. Try using the Euclidean algorithm.

19. Least Common Multiple (LCM)

Find the LCM of two integers. Handle zero and reduce the risk of overflow.

20. Perfect Number

Determine whether a positive integer equals the sum of its positive proper divisors. Example: 6 = 1 + 2 + 3.

21. Strong Number

Check whether a number equals the sum of the factorials of its digits. Example: 145 = 1! + 4! + 5!.

22. Power Without std::pow()

Calculate a^b using a loop, where b is a non-negative integer. Consider overflow and the case 0^0.

23. Count Even and Odd Digits

Count the even and odd digits in an integer. Example: 123456 → 3 even, 3 odd.

24. Find the First and Last Digit

Extract the first and last digits of a non-negative integer without converting it to a string.

Part D — More Number Challenges

25. Swap Two Numbers

Swap two variables, first using a temporary variable and then without one.

26. Optimized Prime Check

Improve your prime checker to test divisors only up to √n. Explain why it works.

27. Print All Primes in a Range

Print every prime number between two given integers.

28. Prime Factorization

Print the prime factors of a positive integer, including repeated factors. Example: 12 → 2 2 3.

29. Convert Decimal to Binary

Convert a non-negative decimal integer to binary without using a conversion library.

30. Convert Binary to Decimal

Convert a binary number represented as digits into its decimal value. Validate that the input contains only 0 and 1.

31. Power of Two

Determine whether a positive integer is a power of two. Bonus: use a bitwise operation.

32. Digital Root

Repeatedly sum the digits until a single digit remains. Example: 9875 → 2.

Part E — Nested Loops and Patterns

33. Right-Angled Star Triangle

For n = 4, print the following pattern:

```
*
**
***
****
```

34. Inverted Star Triangle

For n = 4, print the following pattern:

```
****
***
**
*
```

35. Number Triangle

For n = 4, print the following pattern:

```
1
12
123
1234
```

36. Repeated-Number Triangle

For n = 4, print the following pattern:

```
1
22
333
4444
```

37. Pyramid Pattern

For n = 4, print the following pattern:

```
*
***
*****
*******
```

38. Diamond Pattern

Print a symmetric diamond made of stars for a given height. Define whether the height is odd or even and handle it consistently.

39. Floyd's Triangle

For n = 4, print the following pattern:

```
1
2 3
4 5 6
7 8 9 10
```

40. Hollow Square Pattern

For n = 4, print the following pattern:

```
****
* *
* *
****
```

Completion checklist

Mark each question as complete when your solution passes your own tests. Aim to explain your approach and its edge cases before moving on.

■ Q1	■ Q2	■ Q3	■ Q4	■ Q5
■ Q6	■ Q7	■ Q8	■ Q9	■ Q10
■ Q11	■ Q12	■ Q13	■ Q14	■ Q15
■ Q16	■ Q17	■ Q18	■ Q19	■ Q20
■ Q21	■ Q22	■ Q23	■ Q24	■ Q25
■ Q26	■ Q27	■ Q28	■ Q29	■ Q30
■ Q31	■ Q32	■ Q33	■ Q34	■ Q35
■ Q36	■ Q37	■ Q38	■ Q39	■ Q40